

Networking & Cybersecurity Track
Simulated Employment Program

Lab Report:
Working with Text Files in CLI

Prepared by:
Kudzaishe Majeza
Junior Network Engineer – VxLEARN Networks

Mentor:
Titus Majeza
Senior Network Engineer

Date: 05 March 2026

OBJECTIVES & REQUIRED RESOURCES

Lab Objectives

Part 1 Graphical Text Editors — using SciTE from the GUI and from the terminal.	Part 2 Command Line Text Editors — editing files with GNU nano.	Part 3 Configuration Files — locating, editing, and applying config changes.
---	--	--

Required Resources

Cisco CyberOps Workstation Virtual Machine accessed via VirtualBox or VMware. Login credentials: analyst / cyberops.

PART 1 — GRAPHICAL TEXT EDITORS

Text editors are among the oldest applications in computing. Linux provides many options ranging from graphical editors to command-line-only tools. The Cisco CyberOps Workstation VM includes SciTE as its graphical editor small, fast, and sufficient for this course.

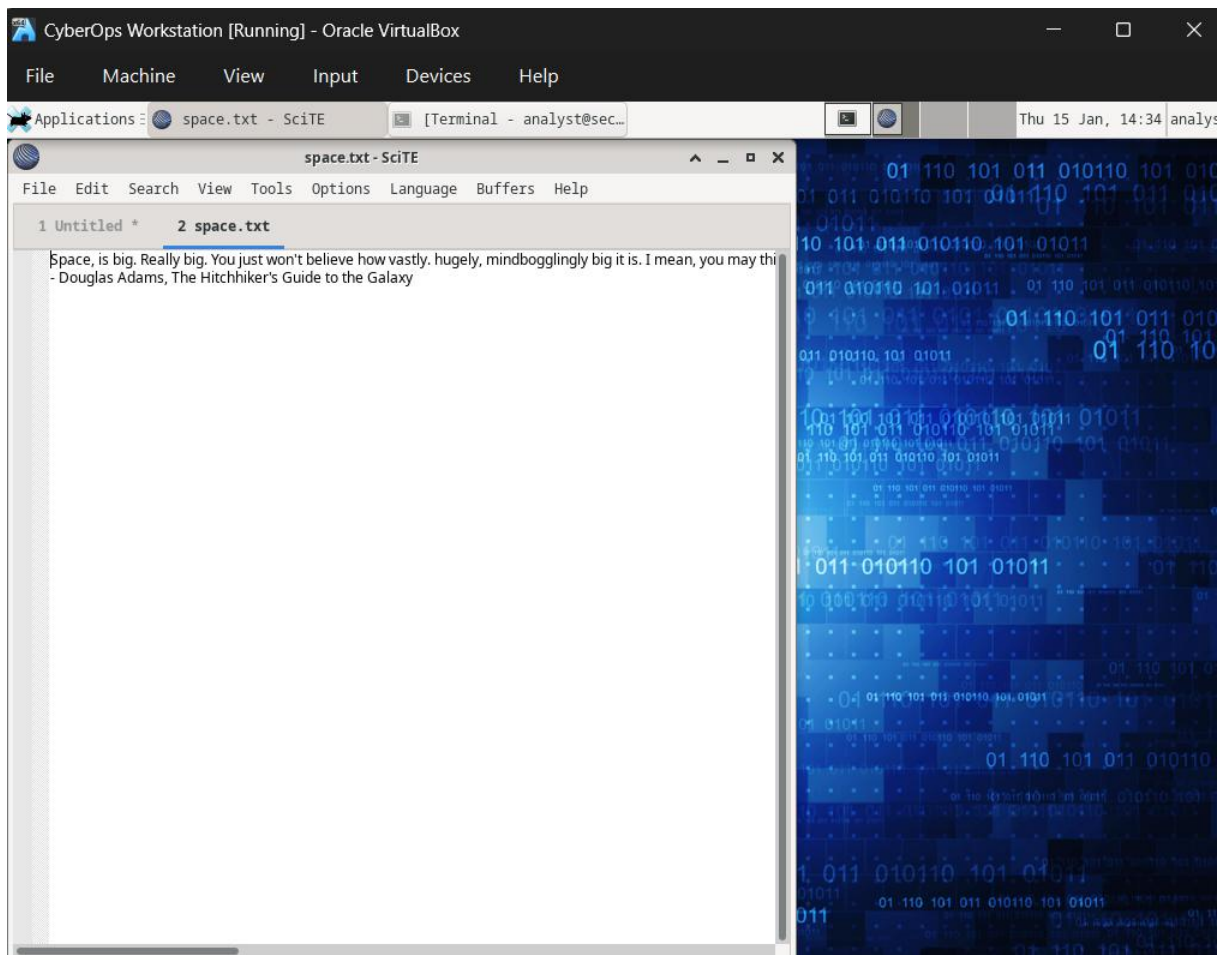
Step 1 — Opening SciTE from the GUI

1. Log in to the CyberOps VM as the user analyst with the password cyberops.
2. Navigate to Applications > CyberOPS > SciTE on the top menu bar to launch the editor.
3. Type or paste the following text into the main work area:

"Space, is big. Really big. You just won't believe how vastly, hugely, mindbogglingly big it is. I mean, you may think it's a long way down the road to the chemist, but that's just peanuts to space."

— Douglas Adams, The Hitchhiker's Guide to the Galaxy

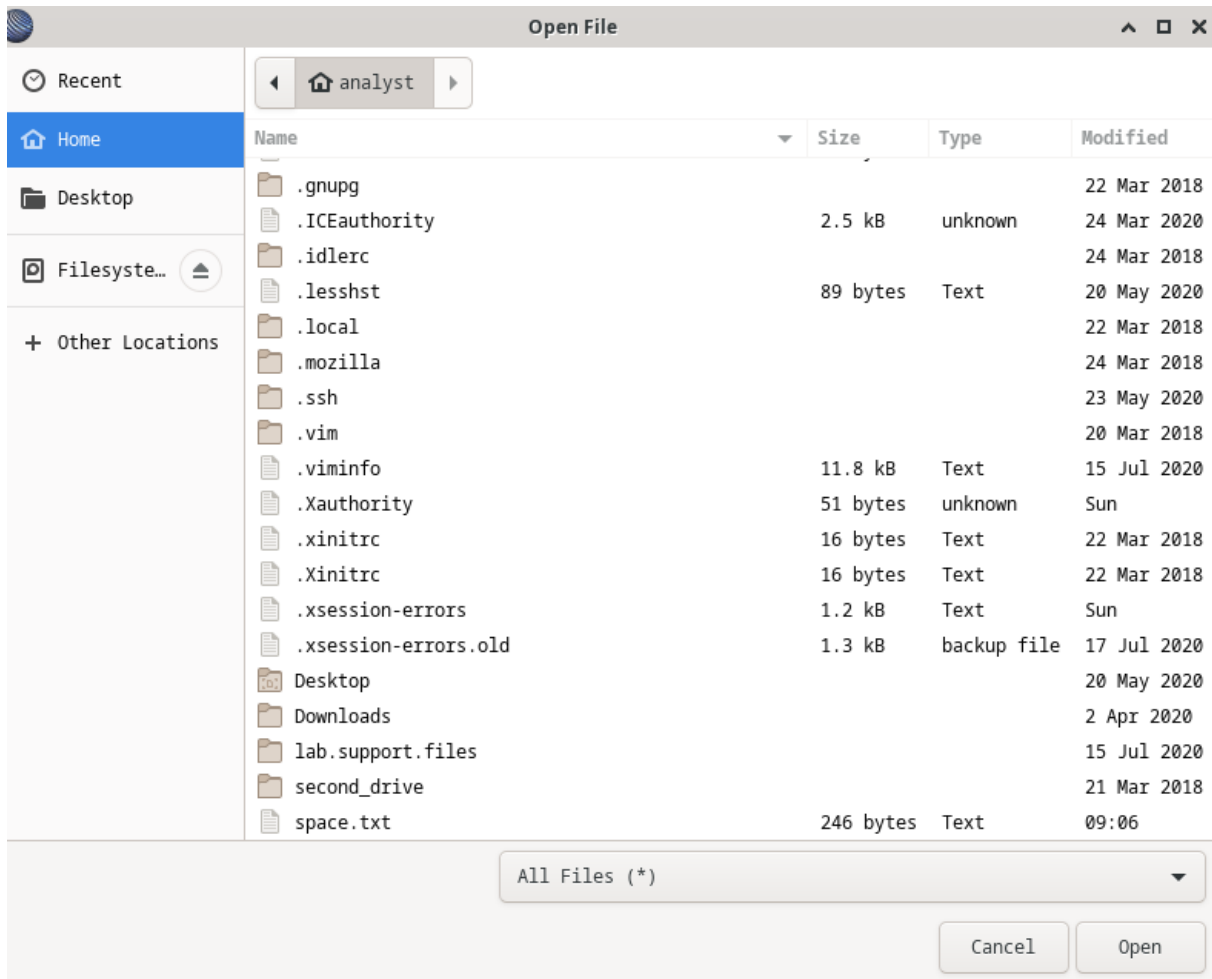
4. Click File > Save. SciTE defaults to /home/analyst. Name the file space.txt and click Save.
5. Close SciTE, then reopen it. Click File > Open... and attempt to find space.txt.



Q: Could you immediately find space.txt after reopening SciTE?

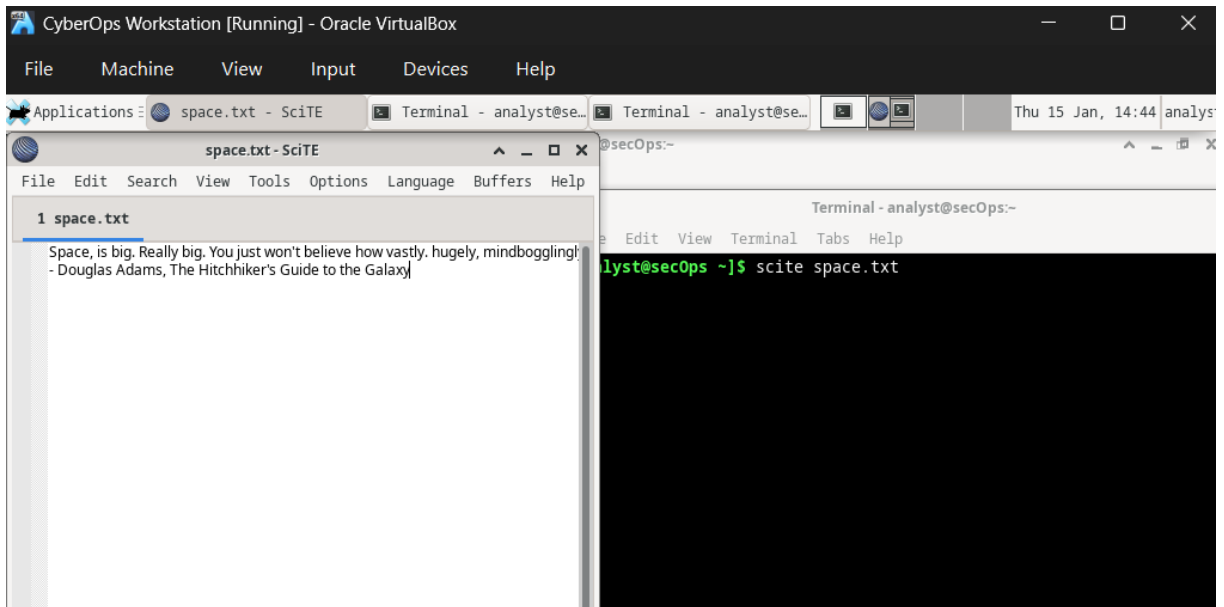
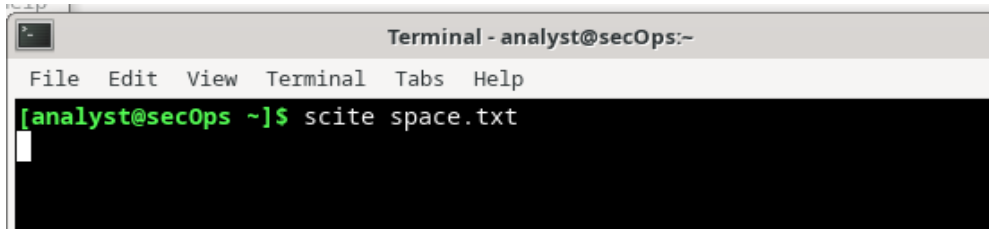
A: No. Although SciTE is looking in the correct directory (/home/analyst), space.txt is not displayed by default because SciTE filters files by known extensions and .txt is not among them. To display it, the file-type dropdown at the bottom of the Open File dialog must be changed to "All Files (*)".

6. Change the file-type filter to All Files (*). space.txt now appears. Select it and click Open.
7. Close space.txt when finished.



Step 2 — Opening SciTE from the Terminal

Click the terminal icon in the Dock. Type `ls` to confirm `space.txt` is in the current directory, then launch SciTE with the file pre-loaded:



Q: Why is the prompt not shown in the terminal after running `scite space.txt`?

A: When `scite space.txt` is run without backgrounding it (no `&` suffix), the shell process blocks and waits for SciTE to finish. The shell prompt does not reappear until SciTE is closed or the process is interrupted with `Ctrl+C`. The terminal is occupied by the foreground SciTE process.

PART 2 — COMMAND LINE TEXT EDITORS

Command-line text editors are essential for administering remote Linux servers via SSH where no graphical interface is available. The CyberOps VM includes GNU nano as the primary command-line editor. Another popular editor is vim (an updated vi), which has a steeper learning curve but is extremely powerful and available on virtually every Linux distribution.

Opening and Navigating nano

Open space.txt with nano from the terminal:



```
Terminal - analyst@secOps:~  
File Edit View Terminal Tabs Help  
GNU nano 4.9.2 space.txt  
Space, is big. Really big. You just won't believe how vastly. hugely, mindbogglingly  
- Douglas Adams, The Hitchhiker's Guide to the Galaxy  
[ Read 2 lines ]  
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos  
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_ Go To Line
```

Q: What character does nano use to show a line continues beyond the screen?

A: nano uses the dollar sign (\$) at the right-hand edge of the terminal to indicate that the current line extends beyond the visible screen boundary. Use the End key to jump to the end of the line and Home to return to the beginning.

nano Keyboard Shortcuts Reference

Shortcut	Action
Ctrl+G	Open Help screen
Ctrl+O	Save (Write Out) current file
Ctrl+X	Exit nano (prompts to save if modified)
Ctrl+W	Search (Where Is)
Ctrl+K	Cut current line
Ctrl+U	Paste (Uncut)
Home / End	Jump to start / end of line
Page Up / Down	Scroll one full page

PART 3 — WORKING WITH CONFIGURATION FILES

In Linux everything is treated as a file — including configuration for every application and service. Configuration files are plain text and are edited with any text editor. User-level config files are hidden (dot-prefixed) in the home directory; system-wide service configs live in /etc and require root privileges to modify.

Step 1 — Locating Configuration Files

a. List visible files in the analyst home directory

```
[analyst@secOps ~]$ ls -l
total 20
drwxr-xr-x 2 analyst analyst 4096 May 20 2020 Desktop
drwxr-xr-x 3 analyst analyst 4096 Apr  2 2020 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jul 15 2020 lab.support.files
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 second_drive
-rw-r--r-- 1 analyst analyst 246 Jan 15 09:06 space.txt
```

b. List all files including hidden ones

```
[analyst@secOps ~]$ ls -la
total 124
drwx----- 14 analyst analyst 4096 Jan 15 09:06 .
drwxr-xr-x  3 root    root    4096 Mar 20 2018 ..
-rw-----  1 analyst analyst 1011 Jan 15 15:14 .bash_history
-rw-r--r--  1 analyst analyst  21 Feb  7 2018 .bash_logout
-rw-r--r--  1 analyst analyst  57 Feb  7 2018 .bash_profile
-rw-r--r--  1 analyst analyst  97 Mar 20 2018 .bashrc
-rw-r--r--  1 analyst analyst 141 Feb  7 2018 .bashrc_stock
drwxr-xr-x  8 analyst analyst 4096 Mar 25 2020 .cache
drwxr-xr-x 10 analyst analyst 4096 Jan 11 18:30 .config
drwxr-xr-x  2 analyst analyst 4096 May 20 2020 Desktop
-rw-r--r--  1 analyst analyst  23 Mar 23 2018 .dmrc
drwxr-xr-x  3 analyst analyst 4096 Apr  2 2020 Downloads
drwx-----  3 analyst analyst 4096 Mar 22 2018 .gnupg
-rw-----  1 analyst analyst 2520 Mar 24 2020 .ICEauthority
drwxr-xr-x  2 analyst analyst 4096 Mar 24 2018 .idlerc
drwxr-xr-x  9 analyst analyst 4096 Jul 15 2020 lab.support.files
-rw-----  1 analyst analyst  89 May 20 2020 .lessht
drwxr-xr-x  3 analyst analyst 4096 Mar 22 2018 .local
drwx-----  5 analyst analyst 4096 Mar 24 2018 .mozilla
drwxr-xr-x  2 analyst analyst 4096 Mar 21 2018 second_drive
-rw-r--r--  1 analyst analyst  246 Jan 15 09:06 space.txt
drwx-----  2 analyst analyst 4096 May 23 2020 .ssh
drwxr-xr-x  3 analyst analyst 4096 Mar 20 2018 .vim
-rw-----  1 analyst analyst 11848 Jul 15 2020 .viminfo
-rw-----  1 analyst analyst  51 Jan 11 18:19 .Xauthority
-rw-r--r--  1 analyst analyst  16 Mar 22 2018 .xinitrc
-rw-r--r--  1 analyst analyst  16 Mar 22 2018 .Xinitrc
-rw-----  1 analyst analyst 1471 Jan 15 14:40 .xsession-errors
-rw-----  1 analyst analyst 1298 Jul 17 2020 .xsession-errors.old
```


c. View the user-level .bashrc

```
[analyst@secOps ~]$ cat .bashrc
export EDITOR=vim

PS1='\[\e[1;32m\][\u@\h \W]\$\[\e[0m\] '
alias ls="ls --color"
alias vi="vim"
```

d. View the system-wide bash.bashrc

```
Terminal - analyst@secOps
[analyst@secOps ~]$ cat /etc/bash.bashrc

#
# /etc/bash.bashrc
#
# If not running interactively, don't do anything
[[ $- != *i* ]] && return

PS1='\u@\h \W]\$\ '

case ${TERM} in
    xterm*|rxvt*)
        PROMPT_COMMAND='printf "\033]0;%s@%s:%s\007" ...'
        ;;
    *)
        ;;
esac

[ -r /usr/share/bash-completion/bash_completion ] && \
. /usr/share/bash-completion/bash_completion
```

Q: Why are user config files saved in the home directory rather than /etc?

A: User application configuration files are stored in the home directory (~) because each user has write permission only to their own home directory, not to /etc. This enforces the principle of least privilege: a regular user can customize their own environment without the ability to accidentally or maliciously alter system-wide settings that affect all users. Each user can also maintain independent settings for the same application.

Step 2 — Editing and Saving Configuration Files

In this step the shell prompt color is changed from green → red → yellow by modifying the ANSI color code in .bashrc.

Bash Prompt Color Codes Used in This Lab

Code	Color	Used In
32	Green (default)	Initial .bashrc setting
31	Red	After SciTE edit (Step 2a)
33	Yellow	After nano edit (Step 2h)

Editing .bashrc with SciTE (GUI) — Change green to red

- Open SciTE via Applications > CyberOPS > SciTE, then File > Open...
- Set the file filter to All Files (*) and select .bashrc from /home/analyst.

10. Locate the line containing `PS1='\[\e[1;32m\]...` and change 32 to 31.



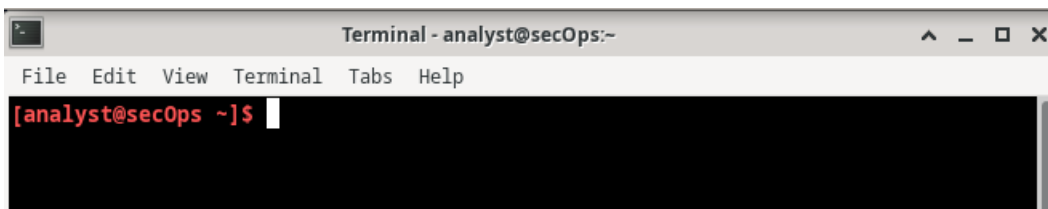
```
.bashrc - SciTE
File Edit Search View Tools Options Language Buffers Help

1 .bashrc
export EDITOR=vim

PS1='\[\e[1;32m\][\u@\h \W]\$'\[\e[0m\] '
alias ls='ls -color'
alias vi='vim'
```

11. Save with File > Save and close SciTE.

12. Open a new terminal window — the prompt appears in red.



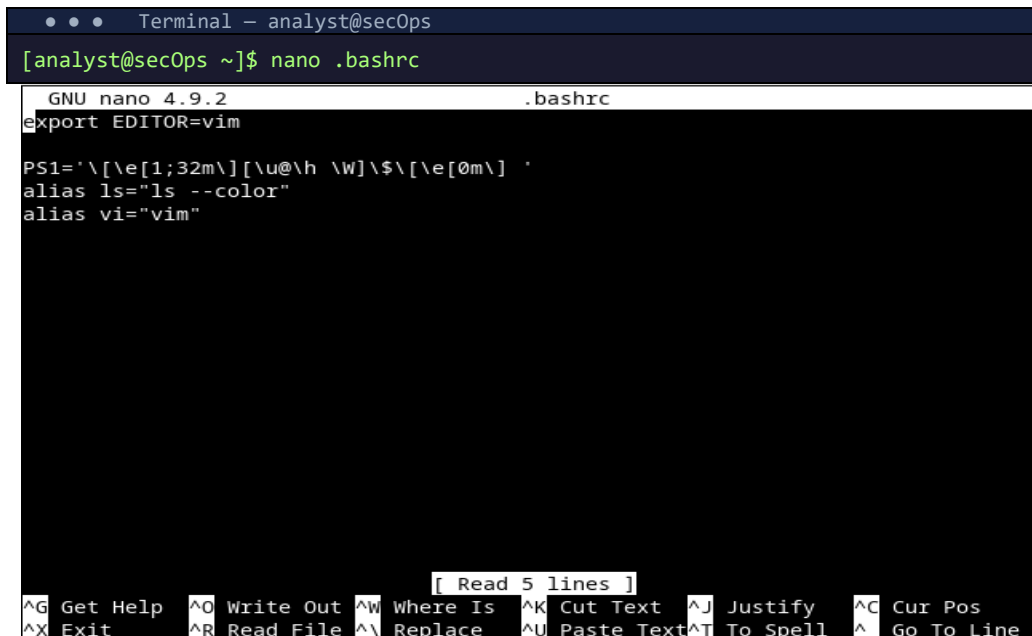
```
Terminal - analyst@secOps:~
File Edit View Terminal Tabs Help

[analyst@secOps ~]$
```

Q: Did the already-open terminal window also change to red? Explain.

A: No. The existing terminal window retained the green prompt because `.bashrc` is only sourced (executed) when a new interactive Bash shell starts. The running terminal session had already loaded its configuration at startup, so it is unaffected by the saved change. Only newly opened terminal windows — which start a fresh Bash session — will read the updated `.bashrc` and display the red prompt.

Editing `.bashrc` with nano (CLI) — Change red to yellow



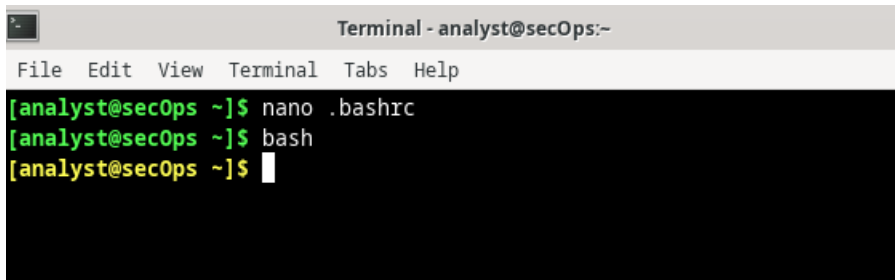
```
Terminal - analyst@secOps
[analyst@secOps ~]$ nano .bashrc

GNU nano 4.9.2 .bashrc
export EDITOR=vim

PS1='\[\e[1;32m\][\u@\h \W]\$'\[\e[0m\] '
alias ls='ls -color'
alias vi='vim'

[ Read 5 lines ]
^G Get Help  ^O Write Out  ^W Where Is   ^K Cut Text   ^J Justify    ^C Cur Pos
^X Exit      ^R Read File  ^\ Replace   ^U Paste Text ^T To Spell   ^_ Go To Line
```

13. Change 31 to 33 (yellow).
14. Press Ctrl+X, then Y, then Enter to save as .bashrc.
15. Run bash in the same terminal to reload the configuration — the prompt now appears in yellow.

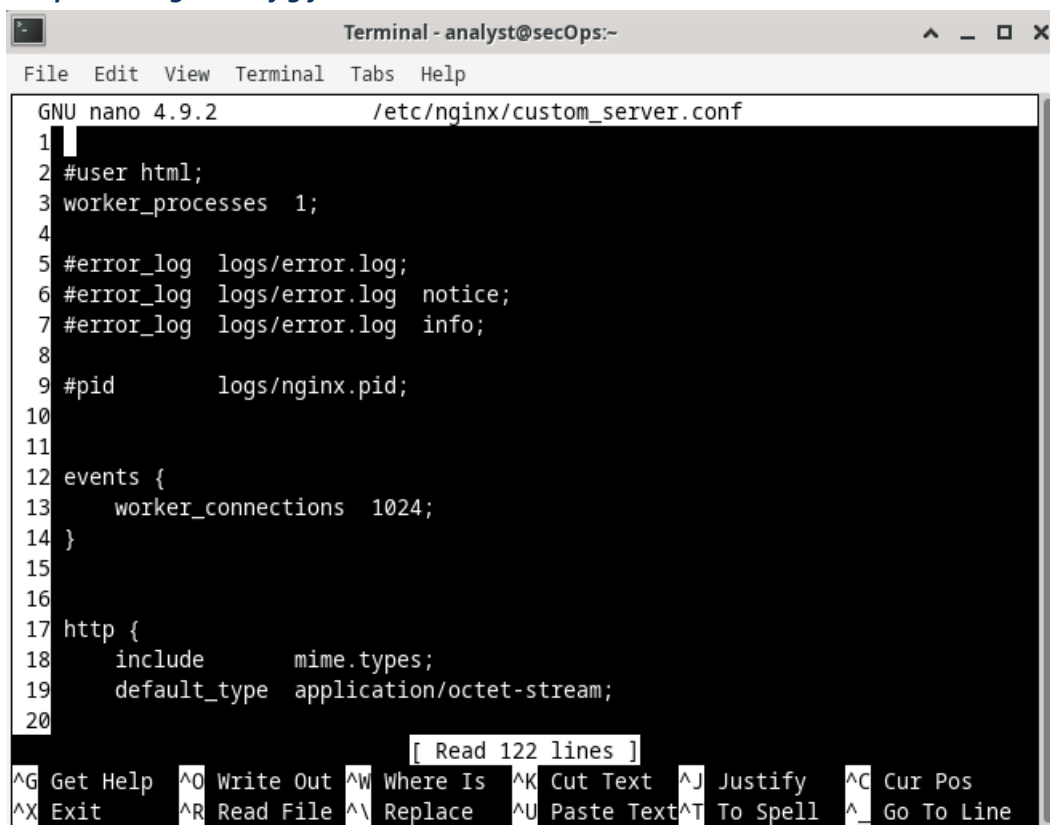


```
Terminal - analyst@secOps:~  
File Edit View Terminal Tabs Help  
[analyst@secOps ~]$ nano .bashrc  
[analyst@secOps ~]$ bash  
[analyst@secOps ~]$
```

Step 3 — Editing Configuration Files for Services (nginx)

nginx is a lightweight web server installed on the CyberOps VM. Its configuration file is at /etc/nginx/custom_server.conf. Because this is a system file, sudo is required to edit it.

a. Open the nginx config file in nano with line numbers



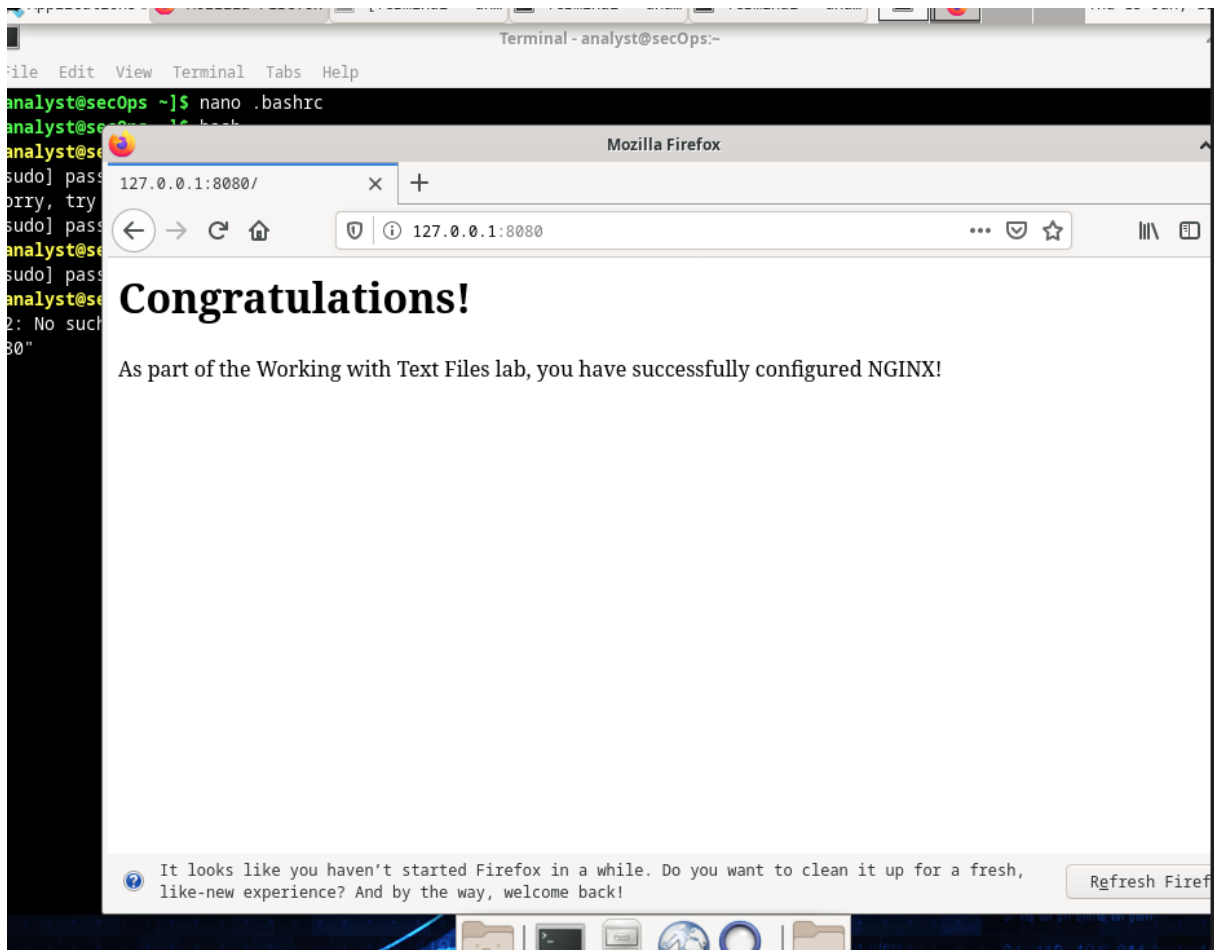
```
Terminal - analyst@secOps:~  
File Edit View Terminal Tabs Help  
GNU nano 4.9.2 /etc/nginx/custom_server.conf  
1  
2 #user html;  
3 worker_processes 1;  
4  
5 #error_log logs/error.log;  
6 #error_log logs/error.log notice;  
7 #error_log logs/error.log info;  
8  
9 #pid logs/nginx.pid;  
10  
11  
12 events {  
13     worker_connections 1024;  
14 }  
15  
16  
17 http {  
18     include mime.types;  
19     default_type application/octet-stream;  
20  
[ Read 122 lines ]  
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos  
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_ Go To Line
```

b–d. Configuration Changes Made

Line	Original	New Value	Purpose
39	listen 81;	listen 8080;	Change HTTP listen port to 8080
47	root /usr/share/nginx/html;	root /usr/share/nginx/html/text_ed_lab;	Serve lab-specific web content

e–f. Save, start nginx, and browse to the lab page

```
[analyst@secOps ~]$ sudo nginx -c custom_server.conf
[sudo] password for analyst:
[analyst@secOps ~]$ 2026/01/15 18:17:19 [error] 11232#11232: *1 open() "/usr/share/nginx/html/text_ed_lab/favicon.ico" failed
(2: No such file or directory), client: 127.0.0.1, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "127.0.0.1:8080"
2026/01/15 18:18:11 [error] 11232#11232: *2 open() "/usr/share/nginx/html/text_ed_lab/favicon.ico" failed (2: No such file or
directory), client: 127.0.0.1, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "127.0.0.1:8080"
```



Q: What is the error message in the terminal referring to?

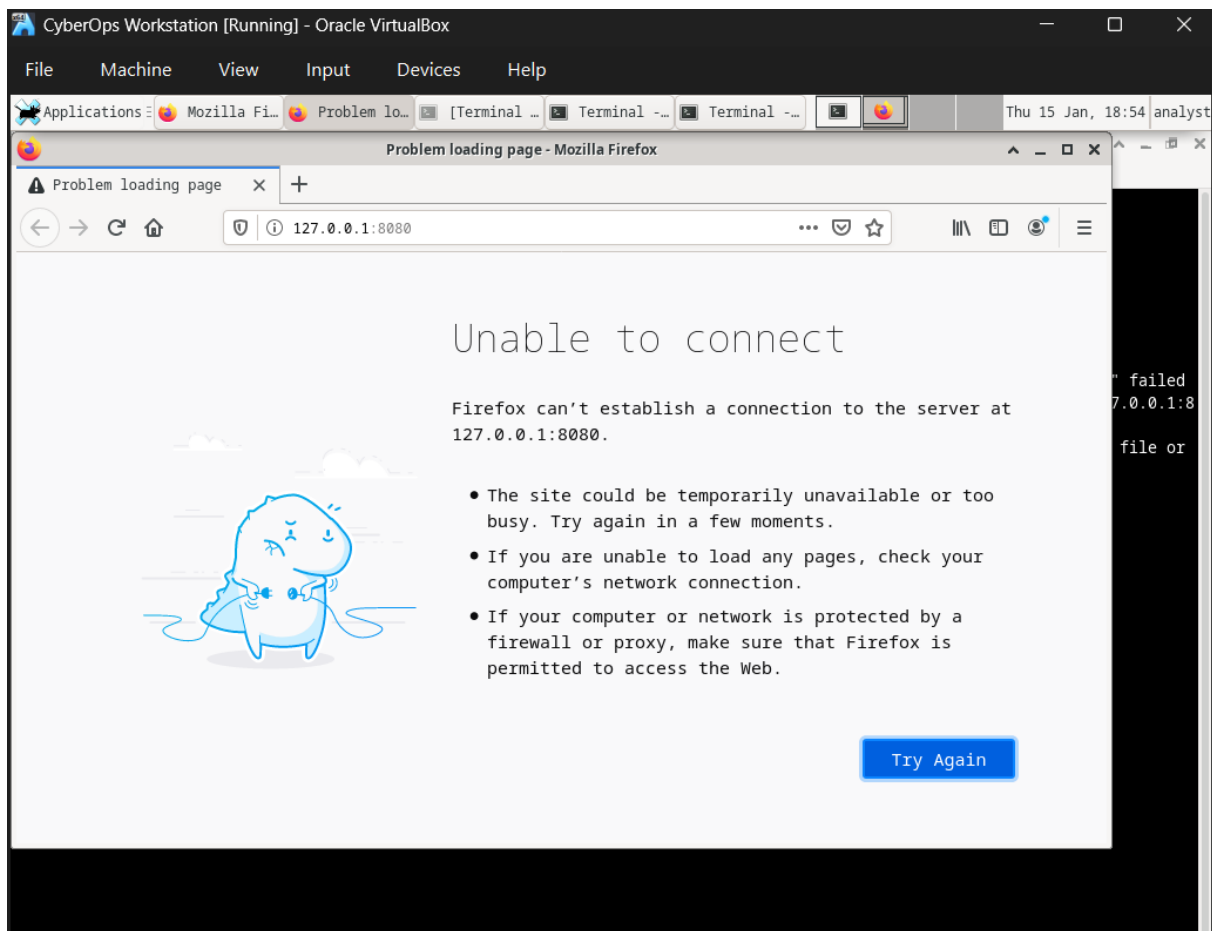
A: After nginx processes the browser's HTTP request, the terminal log may show a 404 Not Found entry for a resource such as favicon.ico. This is expected — the browser automatically requests the site's icon file and since none exists in /usr/share/nginx/html/text_ed_lab/, nginx logs a 404 error for that resource. The main index.html page itself loads correctly.

g. Stop the nginx server

```
Terminal - analyst@secOps
[analyst@secOps ~]$ sudo pkill nginx
# nginx process terminated; port 8080 no longer served
```

Q: Does the web page appear after stopping nginx?

A: No. After running `sudo pkill nginx`, the nginx service is stopped and no longer listens on port 8080. When Firefox cache is cleared and the browser is restarted, navigating to 127.0.0.1:8080 displays an "Unable to connect" error because there is no active server process to respond to the HTTP request.



Challenge Question — Editing /etc/nginx/custom_server.conf with SciTE

Q: Can you edit `/etc/nginx/custom_server.conf` with SciTE? Describe the process.

A: Yes, but root privileges are required. The process:

1. Open a terminal.
2. Run: `sudo scite /etc/nginx/custom_server.conf`
3. Enter the analyst password (cyberops) when prompted.
4. SciTE opens the file with write permissions (the full path bypasses the extension filter).
5. Make the desired edits, then File > Save.

Attempting to open and save without sudo will produce a "Permission denied" error because regular users have only read access to files in `/etc`.

REFLECTION — KEY TAKEAWAYS

- **Configuration file locations vary by scope.** User-application configs live in ~ (home directory, hidden with a dot prefix). System services are configured under /etc which requires root access to modify.
- **Extensions are a convention, not a Linux requirement.** Linux does not rely on file extensions to determine file type, but applications such as SciTE may filter by extension in their open dialogs. Using "All Files (*)" bypasses this.
- **Shell sessions must be reloaded to pick up .bashrc changes.** .bashrc is sourced only at the start of a new interactive shell. Run bash or source ~/.bashrc to apply changes to the current session.
- **Permissions are the most common obstacle when editing service configs.** Always use sudo when editing files in /etc. Failing to do so will produce a "Permission denied" error at save time.
- **Services must be restarted to apply configuration changes.** Editing a config file does not reload the service. nginx must be started fresh with the updated config path before changes take effect.
- **nano is essential for remote / headless environments.** When connected over SSH with no GUI, nano (or vim) is the only way to edit files. Knowing Ctrl+O, Ctrl+X, and Ctrl+W is a fundamental sysadmin skill.

COMMANDS USED IN THIS LAB

Command	Description
scite space.txt	Open SciTE with a specified file from the terminal
sudo scite /etc/nginx/custom_server.conf	Open SciTE as root to edit a system config file
nano space.txt	Open GNU nano with a specified file
sudo nano -l /etc/nginx/custom_server.conf	Open system config in nano as root with line numbers
ls -l	List files with details (no hidden files)
ls -la	List all files including hidden dot-files
cat .bashrc	Display user shell configuration
cat /etc/bash.bashrc	Display system-wide shell configuration
sudo nginx -c custom_server.conf	Start nginx with a custom configuration file
sudo pkill nginx	Stop all running nginx processes
bash	Start a new Bash session (reloads .bashrc)
source ~/.bashrc	Reload .bashrc without opening a new terminal